

WAP TO IMPLEMENT ROUND ROBIN PROCESS SCHEDULING ALGORITHM

CODE:

```
#include <iostream>
#include <vector>
using namespace std;

struct Process {
    int pid;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int waiting_time = 0;
    int turnaround_time = 0;
    bool finished = false;
};

int main() {
    int n, time_quantum;
    cout << "Enter number of processes: ";
    cin >> n;
    vector<Process> processes(n);
    cout << "Enter arrival time and burst time for each process:\n";
    for (int i = 0; i < n; i++) {
        processes[i].pid = i + 1;
        cout << "Process " << i + 1 << " arrival time: ";
        cin >> processes[i].arrival_time;
        cout << "Process " << i + 1 << " burst time: ";
        cin >> processes[i].burst_time;
        processes[i].remaining_time = processes[i].burst_time;
    }
    cout << "Enter time quantum: ";
    cin >> time_quantum;
    int t = 0;
    int completed = 0;
    int i = 0;
    while (completed < n) {
        bool found = false;
        for (int j = 0; j < n; j++) {
            i = (i + j) % n;
            if (processes[i].arrival_time <= t && processes[i].remaining_time > 0) {
                found = true;

                if (processes[i].remaining_time > time_quantum) {
                    t += time_quantum;
                    processes[i].remaining_time -= time_quantum;
                } else {
                    t += processes[i].remaining_time;
                    processes[i].waiting_time = t - processes[i].arrival_time - processes[i].burst_time;
                    processes[i].remaining_time = 0;
                    processes[i].finished = true;
                    completed++;
                }
            }
        }
    }
}
```

```

        i = (i + 1) % n;
        break;
    }
}

if (!found) {
    t++;
}
}

for (int k = 0; k < n; k++) {
    processes[k].turnaround_time = processes[k].burst_time + processes[k].waiting_time;
}

cout << "\nProcess\tArrival\tBurst\tWaiting\tTurnaround\n";
float total_wt = 0, total_tat = 0;
for (int k = 0; k < n; k++) {
    cout << "P" << processes[k].pid << "\t" << processes[k].arrival_time << "\t"
        << processes[k].burst_time << "\t" << processes[k].waiting_time << "\t"
        << processes[k].turnaround_time << "\n";

    total_wt += processes[k].waiting_time;
    total_tat += processes[k].turnaround_time;
}

cout << "\nAverage waiting time = " << total_wt / n;
cout << "\nAverage turnaround time = " << total_tat / n << "\n";
return 0;
}

```

Output:

```

PS E:\Normal Prog\Programming\C++\OS 4_sem> cd "e:\Normal Prog\
Enter number of processes: 4
Enter arrival time and burst time for each process:
Process 1 arrival time: 0
Process 1 burst time: 5
Process 2 arrival time: 3
Process 2 burst time: 12
Process 3 arrival time: 5
Process 3 burst time: 3
Process 4 arrival time: 8
Process 4 burst time: 9
Enter time quantum: 2

Process Arrival Burst   Waiting Turnaround
P1      0      5      6      11
P2      3     12     14     26
P3      5      3      6      9
P4      8      9     10     19

Average waiting time = 9
Average turnaround time = 16.25
PS E:\Normal Prog\Programming\C++\OS 4_sem> 

```

WAP TO IMPLEMENT LRU PAGE REPLACEMENT ALGORITHM

Code:

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <list>
#include <sstream>
using namespace std;

int main()
{
    int frames;
    cout << "\n\nEnter the number of page frames: ";
    cin >> frames;
    cin.ignore();

    cout << "Enter the page reference string (space separated): ";
    string line;
    getline(cin, line);
    stringstream ss(line);

    vector<int> pages;
    int page;
    while (ss >> page) {
        pages.push_back(page);
    }

    list<int> lruList;
    unordered_map<int, list<int>::iterator> pageMap;
    int page_faults = 0;

    for (int p : pages) {
        if (pageMap.find(p) == pageMap.end()) {
            page_faults++;
            if ((int)lruList.size() == frames) {
                int last = lruList.back();
                lruList.pop_back();
                pageMap.erase(last);
            }
        } else {
            lruList.erase(pageMap[p]);
        }
        lruList.push_front(p);
        pageMap[p] = lruList.begin();
    }

    cout << "Total page faults: " << page_faults << "\n\n";

    return 0;
}
```

Output Example:

```
PS E:\Normal Prog\Programming\C++\OS 4_sem> cd "e:\Normal Prog\Programming\C++\C
```

```
Enter the number of page frames: 3
```

```
Enter the page reference string (space separated): 7 0 1 2 0 3 0 4 2 3 0 3 2
```

```
Total page faults: 9
```

```
PS E:\Normal Prog\Programming\C++\OS 4_sem> 
```

LINUX CASE STUDY

History:

1. 1991: Linux Kernel Created by Linus Torvalds
Linus Torvalds, a Finnish computer science student, announced the first version of the Linux kernel on August 25, 1991. It started as a personal project to create a free operating system kernel.
2. 1992: Linux Kernel Released Under GPL License
Linux was relicensed under the GNU General Public License (GPL), making it free software and encouraging collaboration and distribution.
3. 1993: First Complete Linux Distributions Appear
Early distributions like Slackware and Debian made Linux more accessible by bundling the kernel with GNU tools, libraries, and applications.
4. 1994: Linux Kernel 1.0 Released
The first official stable version, Linux kernel 1.0, was released, marking a significant step towards production-quality readiness.
5. 1996: Introduction of the Linux Device Drivers and Kernel Development Model
Development of device drivers and modular kernel architecture advanced, supporting more hardware.
6. 1999: Major Corporations Begin Supporting Linux
Companies like IBM, Oracle, and HP started to invest in Linux, giving it commercial credibility and helping enterprise adoption.
7. 2003: Launch of the Linux Foundation
The Linux Foundation was established to promote, protect, and advance Linux and collaborative development.
8. 2005: Adoption of Git Version Control by Linux Kernel Project
Linus Torvalds created Git to handle Linux kernel source code management efficiently, revolutionizing how open-source projects handle version control.
9. 2011: Linux Powers Majority of Web Servers and Supercomputers
Linux became the dominant OS for servers worldwide and supercomputers, demonstrating its scalability and reliability.
10. 2014: Android Based on Linux Becomes Most Popular OS for Mobile
Google's Android OS, built on the Linux kernel, surpassed other mobile operating systems globally.

Features:

1. Portable And Light weight.
2. Open Source
3. Multi User
4. Multiprogramming
5. Shell
6. Security

Architecture:

- Hardware layer
- Kernel module
- Shell layer
- Application/Utilities layer

Hardware layer:

This layer directly interacts with the physical components of the system such as the CPU, memory, hard disk, I/O devices, etc. It is the foundation upon which the entire Linux operating system runs. The kernel uses device drivers to communicate with this hardware.

Kernel Module:

This is the core component of the Linux operating system. It manages system resources and acts as a bridge between hardware and software. The kernel handles:

- Process management
- Memory management
- File systems
- Device control via drivers
- System calls

It operates in privileged mode and ensures that all applications access hardware in a safe and controlled way.

Fun fact:(The Linux kernel is so modular that it can dynamically load and unload **kernel modules** at runtime — without rebooting or affecting other parts of the system. These modules are usually compiled as **separate binary files** (. ko – kernel object files), meaning a single binary file can add entire features like device drivers, filesystems, or network protocols to the running kernel!)

Shell Layer:

The shell is an interface between the user and the kernel. It takes user commands (typed in a terminal or CLI), interprets them, and passes them to the kernel for execution. Shells can be:

- Command-line shells (e.g., Bash)
- Graphical shells (e.g., GNOME, KDE)

It also allows scripting and automation of tasks using shell scripts.

Application/Utilities Layer:

This layer consists of the programs and utilities that users interact with. These include:

- User applications (e.g., text editors, browsers, compilers)
- System utilities (e.g., date, passwd, top, df, etc.)
- Security tools and configuration tools

These programs rely on the shell and kernel to perform tasks and provide user-level functionality.

Process Management:

A process refers to a program in execution; it is a running instance of a program. The processes have an id (PID) and also have a user id (UID) and Linux additionally has personality associated with the process. Personality of a process is used by emulation libraries to be able to cater to arrangement of implementation. Usually a fork/child process inherits the personality of the parent's environment. In Linux, two vectors define a process, an argument vector(it has the command line argument used by the process) and environment vector(process name, id). Linux treats processes and threads as similar representations.

The process can be created by either using system() function which is simple but inefficient and using fork() and exec() function which is advanced but is highly efficient.

States of a process in Linux:

Running: It means the process is either running or it is ready to run.

Waiting: It means the process is waiting for an event to occur or for a system resource.

Stopped: In it, the process is stopped, usually by receiving a signal.

Zombie: here a process is dead, but there is still an entry on the process table.

Inter-process Communication:

In Linux, Inter-process Communication (IPC) refers to mechanisms that allow processes to exchange data and signals, enabling coordination and cooperation between processes.

Types of IPC Mechanisms:

a) Pipes (|)

Unidirectional communication between related processes (typically parent-child).

E.g., `ls | grep txt`

b) Named Pipes (FIFOs)

Like pipes but with a name in the filesystem (`mkfifo`).

Allows communication between unrelated processes.

c) Message Queues

Processes can send and receive structured messages via a queue.

Offers more flexibility than pipes (e.g., message priority)

d) Shared Memory

Fastest IPC method.

Multiple processes can access the same memory region.

Must be synchronized using semaphores or mutexes to avoid race conditions.

e) Semaphores

Used to control access to shared resources.

Acts like a locking mechanism to prevent race conditions.

f) Signals

Used to notify a process that a particular event has occurred (e.g., `SIGINT`, `SIGKILL`).

Can interrupt a process to execute a signal handler.

Memory Management:

Two major components of memory management of Linux are:

- Page Management
- Virtual Memory Management

Page Management:

Pages are usually of size which is power of 2. Given the main memory Linux allocates a group of pages using a buddy system. The allocation is the responsibility of software called 'Page Allocator'. It is responsible for both allocation and, as well as, freeing the memory. The basic memory allocator uses a buddy heap which allocates a contiguous area of size 2^n the required memory with the minimum n by successive generations of "buddies" of equal sizes.

Virtual Memory Management:

Virtual Memory Management is a technique used by Linux and other modern operating systems to provide each process with the illusion of having its own large, private memory space, even though physical RAM is limited. This is achieved by mapping virtual addresses to physical memory using the Memory Management Unit (MMU) and data structures like page tables. Memory is divided into

fixed-size blocks called pages, which are loaded into RAM only when needed — a concept known as demand paging. If a required page is not in memory, a page fault occurs, and the operating system retrieves it from disk. To handle memory overflow, the system uses swap space, where inactive pages are temporarily stored. Virtual memory also provides strong protection, ensuring that processes are isolated and cannot interfere with each other's memory. Additionally, techniques like copy-on-write (COW) allow efficient memory sharing between processes until a write operation occurs. Overall, virtual memory management improves system efficiency, multitasking, and security.

File System:

In Linux, the file system is basically a big hierarchy of directories (or folders) that organize and hold files. One interesting thing is that Linux treats directories themselves as special files—they just happen to contain the names of other files. So in Linux, pretty much everything is a file: programs, services, text documents, images, and even hardware devices.

The system also defines naming rules—like what characters can be used in file names and how long they can be—known as namespaces.

All files and directories are arranged in a tree-like structure starting from the root directory `/`. Think of `/` as Linux's version of the "C:\\" drive in Windows, but instead of multiple drives, Linux has one unified structure.

Short overview:

`/` = The root directory, where it all begins.

`/bin` = Contains essential command-line programs you need to run the system.

`/boot` = Holds files necessary to boot up your computer.

`/dev` = Special files that represent hardware devices like printers and disks.

`/lib` = Shared libraries and kernel modules needed by the system.

`/tmp` = Temporary files go here—usually deleted after reboot.

`/etc` = Yep, it's a real directory (despite sounding like "etcetera") and holds system-wide configuration.

`/home` = Where all users keep their personal files and settings.

`/var` = Stores variable data such as logs and caches, etc.

Linux's file system design emphasizes simplicity, consistency, and flexibility, making it a powerful tool for users and developers alike.

Conclusion:

This case study shows how Linux grew from a small project in 1991 into one of the most powerful and widely used operating systems today. Its open-source nature and GPL license have encouraged worldwide collaboration and rapid growth. With features like multi-user support, multitasking, strong security, efficient process and memory management, and a flexible file system, Linux has proven to be reliable, secure, and adaptable. Its layered architecture makes it suitable for everything from personal computers to servers, supercomputers, and mobile devices. Overall, Linux remains an important and future-ready operating system in the modern computing world.